

Error de compilación al hacer push — InventoryHUD.cpp

BattleRoyale · Resumen de diagnóstico y solución · 2026-09-04

1. Síntoma

Cada push a GitHub disparaba un error de compilación en la build de CI. El error se solucionaba históricamente de forma manual e incidental: editando el archivo afectado y dejando o quitando una línea en blanco, sin entender la causa real.

2. Error reportado por el compilador

```
0>InventoryHUD.cpp(50): Error C2039 : 'GetObjectW': is not a member of 'TScriptInterface<IPickupObject>'
0>TargetDataPickupObject.h(22): Reference : see declaration of 'TScriptInterface<IPickupObject>'
0>InventoryHUD.cpp(50): Error C2737 : 'object': const object must be initialized
0>InventoryHUD.cpp(50): Error C3536 : 'object': cannot be used before it is initialized
0>InventoryHUD.cpp(51): Error C2672 : 'Cast': no matching overloaded function found
0>Linker.h(614): Reference : could be 'T *Cast(FLinker *)'
```

Línea de código afectada (InventoryHUD.cpp, línea 50):

```
if(const auto object = equippedObject.GetObject())
    pickableObject = Cast<APickableObjectBase>(object);
```

3. Diagnóstico

No es un bug en el código de gameplay, sino una **contaminación de macros de Windows.h** dentro del Unity Build de Unreal:

- **GetObject** es una macro de la Windows GDI (definida en wingdi.h) que el preprocesador expande a **GetObjectA** o **GetObjectW** según esté definido UNICODE.
- Unreal, en Unity Build, concatena varios .cpp en un mismo archivo de traducción para acelerar la compilación.
- El módulo depende de **OnlineSubsystemSteam** (declarado en MultiplayerSessions.Build.cs). Los headers internos del motor para Steam filtran esa macro de Windows sin "esconderla" correctamente con HideWindowsPlatformTypes.h.
- Si InventoryHUD.cpp cae en el mismo bloque unity que ese header contaminado, el preprocesador reescribe literalmente **equippedObject.GetObject()** como **equippedObject.GetObjectW()**, un método que no existe en TScriptInterface. De ahí la cascada de errores C2039 / C2737 / C3536 / C2672.
- El truco de "tocar una línea en blanco" funcionaba porque cambia cómo UBT agrupa los archivos en el unity build, y el archivo dejaba de caer junto al header contaminante — pero es un efecto colateral, no una solución real. Por eso el error reaparecía con otra combinación de compilación en CI.

4. Solución aplicada

En vez de depender del azar del orden de compilación, se neutraliza la macro justo antes de usarla, en **InventoryHUD.cpp**:

```
APickableObjectBase* pickableObject = nullptr;

// GetObject collides with the Windows GDI macro (GetObjectA/W) leaked into this
// unity build block via OnlineSubsystemSteam headers; undef it before calling.
#undef GetObject
if(const auto object = equippedObject.GetObject())
    pickableObject = Cast<APickableObjectBase>(object);
```

5. ¿Qué hace #undef?

Antes de que el compilador de C++ vea el código, el **preprocesador** hace sustituciones de texto puro, sin entender sintaxis ni tipos. Windows.h define algo como:

```
#define GetObject GetObjectW // (o GetObjectA, según UNICODE)
```

Es decir: "cualquier aparición del texto *GetObject* en el código, reemplázala por *GetObjectW*". El preprocesador no sabe que en este caso *GetObject* es el nombre de un método de TScriptInterface — solo ve texto y lo sustituye ciegamente.

Al escribir **#undef GetObject** justo antes del uso, se elimina esa definición de macro para el resto del archivo (o del bloque unity). El preprocesador deja entonces *GetObject* tal cual, y el compilador ve la llamada real al método **GetObject()** de TScriptInterface, que sí existe. El efecto es local: no afecta a otros archivos del proyecto.

6. Alcance y estado

El mismo patrón `.GetObject()` sobre TScriptInterface se usa en otros 7 archivos del proyecto (InventoryComponent.cpp, CombatComponent.cpp, AbilityReload.cpp, PickupSelectorComponent.cpp, InventoryArray.cpp, GameRules.cpp), cualquiera de los cuales podría verse afectado en el futuro por la misma causa si cae en el mismo bloque unity contaminado. Se decidió **no aplicar el fix preventivo** en esos archivos por ahora.

Tras aplicar el cambio en InventoryHUD.cpp, el siguiente push compiló correctamente en CI.

Si el mismo error reaparece en otro archivo, la solución es idéntica: añadir `#undef GetObject` justo antes de la línea que llama a `.GetObject()`.